

Лабораторна робота №5

Студент: Сікора Віктор

Група: ТК-32

Зміст

1. Мета роботи
2. Структура проекту
3. Завдання 1 — Читання пам'яті іншого процесу
 1. 1.1 Процес-ціль (target)
 2. 1.2 Монітор пам'яті (monitor)
 3. 1.3 Демонстрація роботи
4. Завдання 2 — Перехоплення клавіатури
 1. 2.1 Перехоплювач (kbd_hook)
 2. 2.2 Тестовий інжектор подій (inject_events)
 3. 2.3 Демонстрація роботи
5. Використані системні виклики
6. Висновки

1. Мета роботи

Дослідити системні виклики Linux (System Call API) для прямої взаємодії з ядром ОС. Реалізувати дві програми: читання пам'яті іншого процесу через `process_vm_readv(2)` та `/proc/[pid]/mem`, а також глобальне перехоплення клавіатури через підсистему вводу ядра Linux (`/dev/input/eventN`).

2. Структура проєкту

```
Lab5/
├─ Makefile
├─ README.md
├─ task1_proc_mem/
│   ├── target.cpp      – процес-"ціль" (84 рядки)
│   └─ monitor.cpp     – читач пам'яті (153 рядки)
└─ task2_kbd_hook/
    ├── kbd_hook.cpp    – перехоплювач клавіатури (296 рядків)
    └─ inject_events.cpp – тестовий інжектор подій (88 рядків)
```

Збірка виконується командою `make`. Використовується C++17 з прапорцями

```
-std=gnu++17 -Wall -Wextra -O2.
```

Ref: Makefile, рядки 1–8

```
CXX      := g++
CXXFLAGS := -std=gnu++17 -Wall -Wextra -O2

BIN1 := task1_proc_mem/target
BIN2 := task1_proc_mem/monitor
BIN3 := task2_kbd_hook/kbd_hook
BIN4 := task2_kbd_hook/inject_events
```

3. Завдання 1 — Читання пам'яті іншого процесу

Завдання складається з двох програм: **target** (процес-ціль, чию пам'ять читатимемо) та **monitor** (програма, що зчитує значення змінної з адресного простору target).

3.1 Процес-ціль (target.cpp)

Програма target оголошує змінну `monitored_value`, виводить свій PID та адресу цієї змінної, після чого в циклі приймає від користувача нові значення.

Ref: task1_proc_mem/target.cpp, рядки 26–31

```
/* Змінна, за якою стежитиме monitor.
 * volatile — компілятор не оптимізує звертання. */
volatile int monitored_value = 0;

static volatile sig_atomic_t running = 1;
static void handle_sigint(int) { running = 0; }
```

Ключовий системний виклик — `prctl(2)`:

Ref: task1_proc_mem/target.cpp, рядки 38–47

```
/*
 * PR_SET_PTRACER_ANY — системний виклик prctl(2).
 * Дозволяє будь-якому процесу читати пам'ять цього процесу
 * через process_vm_readv(2) / /proc/[pid]/mem навіть при
 * Yama ptrace_scope=1.
 */
if (prctl(PR_SET_PTRACER, PR_SET_PTRACER_ANY, 0, 0, 0) != 0) {
    perror("prctl(PR_SET_PTRACER_ANY)");
}
```

Yama LSM (Linux Security Module) за замовчуванням обмежує доступ до пам'яті інших процесів (`ptrace_scope=1`). Виклик `prctl(PR_SET_PTRACER, PR_SET_PTRACER_ANY)` явно дозволяє будь-якому процесу читати нашу пам'ять.

Вивід PID та адреси:

Ref: task1_proc_mem/target.cpp, рядки 49–55

```
printf("PID          : %d\n", (int)getpid());
printf("&monitored_value : %p\n", (void *)&monitored_value);
printf("Запустіть монітор:\n");
printf("  ./monitor %d %p\n", (int)getpid(), (void *)&monitored_value);
```

Цикл введення:

Ref: task1_proc_mem/target.cpp, рядки 57–79

```
while (running) {
    printf("Введіть нове значення (Ctrl+C для виходу): ");
    fflush(stdout);

    int val;
    int ret = scanf("%d", &val);
    if (ret == EOF) { pause(); break; }
    if (ret != 1) {
        int c;
        while ((c = getchar()) != '\n' && c != EOF) {}
        if (!running) break;
        continue;
    }

    monitored_value = val;
    printf("[target] monitored_value = %d\n\n", monitored_value);
}
```

3.2 Монітор пам'яті (monitor.cpp)

Монітор зчитує значення змінної з адресного простору іншого процесу двома методами та виводить зміни в реальному часі.

Метод А: `process_vm_readv(2)`

Ref: task1_proc_mem/monitor.cpp, рядки 38–55

```
static bool read_via_vm_readv(pid_t pid, unsigned long addr, int *out)
{
    struct iovec local  = { .iov_base = out,          .iov_len = sizeof(int) };
    struct iovec remote = { .iov_base = (void *)addr, .iov_len = sizeof(int) };

    /*
     * process_vm_readv(2):
     *  pid          - цільовий процес
     *  &local, 1    - iovec у НАШОМУ адресному просторі (куди писати)
     *  &remote, 1   - iovec у ЧУЖОМУ адресному просторі (звідки читати)
     *  0            - прапори (зарезервовані)
     *
     * Повертає кількість прочитаних байт або -1 при помилці.
     */
    ssize_t n = process_vm_readv(pid, &local, 1, &remote, 1, 0);
    return n == (ssize_t)sizeof(int);
}
```

`process_vm_readv(2)` — системний виклик (Linux 3.2+), що читає вміст адресного простору іншого процесу напряму через scatter-gather iovec-вектори. Не потребує ptrace-прив'язки. Вимагає однаковий uid або `CAP_SYS_PTRACE`, або `PR_SET_PTRACER_ANY` у цільового процесу.

Метод Б: `/proc/[pid]/mem` + `pread(2)`

Ref: `task1_proc_mem/monitor.cpp`, рядки 58–67

```
static bool read_via_proc_mem(int mem_fd, unsigned long addr, int *out)
{
    /*
     * pread(2): читає sizeof(int) байт з файлового дескриптора mem_fd,
     * починаючи з позиції addr (= адреса у VA-просторі цільового процесу).
     * Не змінює поточну позицію fd.
     */
    ssize_t n = pread(mem_fd, out, sizeof(int), (off_t)addr);
    return n == (ssize_t)sizeof(int);
}
```

`/proc/[pid]/mem` — псевдофайл ядра, що відображає весь віртуальний адресний простір процесу. Зсув у файлі дорівнює віртуальній адресі. `pread(2)` виконує позиційне читання без зміни поточної позиції файлового дескриптора.

Головний цикл моніторингу:

Ref: `task1_proc_mem/monitor.cpp`, рядки 86–148

```
/* Відкриваємо /proc/[pid]/mem (Метод Б) */
char mem_path[64];
snprintf(mem_path, sizeof(mem_path), "/proc/%d/mem", (int)pid);
int mem_fd = open(mem_path, O_RDONLY);

...

while (running) {
    bool ok = false;

    /* Спроба 1: process_vm_readv */
    if (!ok) ok = read_via_vm_readv(pid, addr, &current_value);

    /* Спроба 2: /proc/[pid]/mem */
    if (!ok && mem_fd >= 0) ok = read_via_proc_mem(mem_fd, addr, &current_value);

    if (!ok) {
        fprintf(stderr, "[!] Цільовий процес завершився?\n");
        break;
    }

    if (first_read || current_value != last_value) {
        printf("[monitor] значення: %d → %d\n", last_value, current_value);
        fflush(stdout);
        last_value = current_value;
        first_read = false;
    }

    usleep(POLL_US); /* 100 мс пауза між опитуваннями */
}
```

Монітор спочатку пробує `process_vm_readv` (швидший, без файлового дескриптора), а у разі невдачі — `/proc/[pid]/mem`. Інтервал опитування: 100 мс. При зміні значення змінної виводиться повідомлення з попереднім та новим значенням.

3.3 Демонстрація роботи

Для демонстрації потрібно два термінали:

```
# Термінал 1 (target):  
$ ./task1_proc_mem/target  
PID           : 12345  
&monitored_value : 0x601060  
Запустіть монітор:  
    ./monitor 12345 0x601060  
  
Введіть нове значення: 42  
[target] monitored_value = 42  
  
# Термінал 2 (monitor):  
$ ./task1_proc_mem/monitor 12345 0x601060  
[init] початкове значення = 0  
[monitor] значення: 0 → 42  
[monitor] значення: 42 → 100
```


4. Завдання 2 — Перехоплення клавіатури

Завдання демонструє глобальне перехоплення клавіатурних подій через підсистему вводу ядра Linux. Складається з двох програм: **kbd_hook** (перехоплювач) та **inject_events** (тестовий генератор подій).

4.1 Перехоплювач клавіатури (kbd_hook.cpp)

Linux представляє кожен пристрій вводу як файл `/dev/input/eventN`. Ядро записує туди `struct input_event` при кожному натисканні/відпусканні. Читаючи цей файл, програма отримує **всі** події незалежно від того, яке вікно активне.

Автоматичний пошук клавіатури:

Ref: task2_kbd_hook/kbd_hook.cpp, рядки 141–184

```

static int find_keyboard(char *found_path, size_t path_sz)
{
    DIR *dir = opendir("/dev/input");
    ...
    struct dirent *ent;
    while ((ent = readdir(dir)) != nullptr) {
        if (strncmp(ent->d_name, "event", 5) != 0) continue;

        char path[11 + NAME_MAX + 1];
        snprintf(path, sizeof(path), "/dev/input/%s", ent->d_name);

        int fd = open(path, O_RDONLY | O_NONBLOCK);
        if (fd < 0) continue;

        char name[256] = {};
        get_device_name(fd, name, sizeof(name));

        if (has_key_events(fd)) {
            unsigned char keybits[(KEY_MAX + 7) / 8] = {};
            ioctl(fd, EVIOCGBIT(EV_KEY, sizeof(keybits)), keybits);
            bool has_alpha = (keybits[KEY_A / 8] >> (KEY_A % 8)) & 1;

            if (has_alpha) {
                printf("[пошук] Знайдено клавіатуру: %s\n", path);
                best_fd = fd;
                ...
            }
        }
        close(fd);
    }
    closedir(dir);
    return best_fd;
}

```

Програма перебирає всі `/dev/input/event*`, через `ioctl(EVIOCGBIT)` перевіряє, чи підтримує пристрій клавіатурні події (`EV_KEY`), і чи є серед підтримуваних клавіш літери (`KEY_A..KEY_Z`). Якщо назва пристрою містить "keyboard" — пошук зупиняється.

Перевірка підтримки `EV_KEY`:

Ref: task2_kbd_hook/kbd_hook.cpp, рядки 121–131

```

static bool has_key_events(int fd)
{
    /*
     * ioctl EVIOCGBIT(0, ...) — повертає бітову маску
     * підтримуваних типів подій пристроєм.
     */
    unsigned char evbits[(EV_MAX + 7) / 8] = {};
    if (ioctl(fd, EVIOCGBIT(0, sizeof(evbits)), evbits) < 0)
        return false;
    return (evbits[EV_KEY / 8] >> (EV_KEY % 8)) & 1;
}

```

Основний цикл читання подій:

Ref: task2_kbd_hook/kbd_hook.cpp, рядки 247–291

```
struct input_event ev;
bool shift_held = false;

while (running) {
    ssize_t n = read(kbd_fd, &ev, sizeof(ev));

    if (n < 0) {
        if (errno == EINTR) continue; /* перервано сигналом */
        perror("read");
        break;
    }
    if (n == 0) break; /* EOF (FIFO writer closed) */

    /* Фільтруємо: тільки клавіатурні події (EV_KEY) */
    if (ev.type != EV_KEY) continue;

    /* Відстежуємо стан Shift */
    if (ev.code == KEY_LEFTSHIFT || ev.code == KEY_RIGHTSHIFT)
        shift_held = (ev.value != 0);

    /* Виводимо лише press (1) + repeat (2) */
    if (ev.value != 1 && ev.value != 2) continue;

    const char *name = (ev.code <= KEY_MAX)
        ? KEY_NAMES[ev.code] : nullptr;

    /* Верхній регістр при Shift */
    if (shift_held && strlen(name) == 1
        && name[0] >= 'a' && name[0] <= 'z') {
        upper[0] = (char)(name[0] - 32);
        display = upper;
    }
    printf(" KEY [%3u] %-12s\n", ev.code, display);
}
```

Виклик `read(2)` блокується до появи нової події від ядра. Ядро записує `struct input_event` у `/dev/input/eventN` при кожному натисканні. Структура події:

```
struct input_event {
    struct timeval time; /* час події
    __u16 type;          // EV_KEY, EV_REL, EV_ABS, ...
    __u16 code;          // KEY_A, KEY_ENTER, ...
    __s32 value;         // 0=відпускання, 1=натискання, 2=утримання
};
```

Таблиця відповідності keycode → символ:

Ref: task2_kbd_hook/kbd_hook.cpp, рядки 47–118

```
static const char *KEY_NAMES[KEY_MAX + 1] = {};

static void init_key_names() {
    KEY_NAMES[KEY_ESC] = "ESC";
    KEY_NAMES[KEY_BACKSPACE] = "⌫";
    KEY_NAMES[KEY_TAB] = "⇠";
    KEY_NAMES[KEY_Q]="q"; KEY_NAMES[KEY_W]="w"; KEY_NAMES[KEY_E]="e";
    KEY_NAMES[KEY_R]="r"; KEY_NAMES[KEY_T]="t"; KEY_NAMES[KEY_Y]="y";
    ...
    KEY_NAMES[KEY_ENTER] = "↵ ENTER";
    KEY_NAMES[KEY_SPACE] = "SPACE";
    KEY_NAMES[KEY_UP] = "↑";
    KEY_NAMES[KEY_DOWN] = "↓";
    KEY_NAMES[KEY_LEFT] = "←";
    KEY_NAMES[KEY_RIGHT] = "→";
}
```

4.2 Тестовий інжектор подій (inject_events.cpp)

Для тестування без sudo використовується FIFO-файл. Інжектор записує синтетичні

`struct input_event` у FIFO, а `kbd_hook` читає з нього.

Ref: task2_kbd_hook/inject_events.cpp, рядки 20–40

```
static void emit(int fd, int type, int code, int value)
{
    struct input_event ev{};
    gettimeofday(&ev.time, nullptr);
    ev.type = (__u16)type;
    ev.code = (__u16)code;
    ev.value = (__s32)value;
    write(fd, &ev, sizeof(ev));
}

/* Синтетичне натискання + відпускання */
static void press(int fd, int keycode, useconds_t delay_us = 80000)
{
    emit(fd, EV_MSC, MSC_SCAN, keycode); /* scan-код */
    emit(fd, EV_KEY, keycode, 1); /* PRESS */
    emit(fd, EV_SYN, SYN_REPORT, 0); /* синхро-подія */
    usleep(delay_us);
    emit(fd, EV_KEY, keycode, 0); /* RELEASE */
    emit(fd, EV_SYN, SYN_REPORT, 0);
}
```

Симуляція введення "hello World":

Ref: `task2_kbd_hook/inject_events.cpp`, рядки 52–71

```
/* Симулюємо введення тексту "hello" */
press(fd, KEY_H);
press(fd, KEY_E);
press(fd, KEY_L);
press(fd, KEY_L);
press(fd, KEY_O);
press(fd, KEY_SPACE);

/* Shift + "W" = "W" (верхній перістр) */
emit(fd, EV_KEY, KEY_LEFTSHIFT, 1);
emit(fd, EV_SYN, SYN_REPORT, 0);
press(fd, KEY_W);
emit(fd, EV_KEY, KEY_LEFTSHIFT, 0);
emit(fd, EV_SYN, SYN_REPORT, 0);

press(fd, KEY_O);
press(fd, KEY_R);
press(fd, KEY_L);
press(fd, KEY_D);
press(fd, KEY_ENTER);
```

4.3 Демонстрація роботи

Варіант А — реальна клавіатура (потребує sudo):

```
$ sudo ./task2_kbd_hook/kbd_hook
[пошук] Знайдено клавіатуру: /dev/input/event3 → "AT Translated Set 2 keyboard"
Пристрій : /dev/input/event3
Набирайте текст у будь-якому вікні.

KEY [ 35] h
KEY [ 18] e
KEY [ 38] l
KEY [ 38] l
KEY [ 24] o
```

Варіант Б — тест через FIFO (без sudo):

```
# Термінал 1:
$ mkfifo /tmp/fake_kbd
$ ./task2_kbd_hook/kbd_hook /tmp/fake_kbd

# Термінал 2:
$ ./task2_kbd_hook/inject_events /tmp/fake_kbd
[inject] Sending test key sequence to /tmp/fake_kbd

# Термінал 1 показує:
KEY [ 35] h
KEY [ 18] e
KEY [ 38] l
KEY [ 38] l
KEY [ 24] o
KEY [ 57] SPACE
KEY [ 17] W          ← верхній регістр (Shift)
KEY [ 24] o
KEY [ 19] r
KEY [ 38] l
KEY [ 32] d
KEY [ 28] ↵ ENTER
```

5. Використані системні виклики

Системний виклик	Де використано	Призначення
<code>getpid(2)</code>	target.cpp	Отримати PID поточного процесу
<code>prctl(2)</code>	target.cpp	PR_SET_PTRACER_ANY — дозволити читання пам'яті іншим процесам
<code>process_vm_readv(2)</code>	monitor.cpp	Пряме читання з адресного простору іншого процесу через iovec
<code>open(2)</code>	monitor.cpp, kbd_hook.cpp	Відкрити /proc/[pid]/mem або /dev/input/eventN
<code>pread(2)</code>	monitor.cpp	Позиційне читання з /proc/[pid]/mem без зміни offset
<code>read(2)</code>	kbd_hook.cpp	Читання struct input_event з файлу пристрою
<code>write(2)</code>	inject_events.cpp	Запис синтетичних подій у FIFO
<code>ioctl(2)</code>	kbd_hook.cpp	EVIOCGNAME — назва пристрою; EVIOCGBIT — підтримувані типи подій
<code>close(2)</code>	monitor.cpp, kbd_hook.cpp	Закриття файлового дескриптора
<code>usleep(3)</code>	monitor.cpp, inject_events.cpp	Пауза між опитуваннями / між подіями
<code>signal(2)</code>	target.cpp, monitor.cpp, kbd_hook.cpp	Обробка SIGINT (Ctrl+C) для коректного завершення
<code>fcntl(2)</code>	kbd_hook.cpp	Зміна прапорців fd (вимкнути O_NONBLOCK)

6. Висновки

У ході виконання лабораторної роботи №5 було:

1. Реалізовано читання пам'яті іншого процесу двома методами:

- `process_vm_readv(2)` — прямий доступ до віртуального адресного простору через scatter-gather iovec-вектори (швидший, без файлового дескриптора);
- `/proc/[pid]/mem + pread(2)` — через псевдофайл ядра, де зсув у файлі = віртуальна адреса.

Обидва методи вимагають відповідних привілеїв (`prctl(PR_SET_PTRACER_ANY)` або `CAP_SYS_PTRACE`).

2. Реалізовано глобальне перехоплення клавіатури через підсистему вводу ядра Linux (`/dev/input/eventN`). Програма автоматично знаходить пристрій клавіатури через `ioctl(EVIOCGBIT)`, читає `struct input_event` у блокуючому режимі та відображає натиснуті клавіші з підтримкою Shift.

3. Реалізовано тестовий інжектор подій через FIFO, що дозволяє демонструвати роботу перехоплювача без привілеїв root.

4. Досліджено 12 системних викликів Linux: `getpid`, `prctl`, `process_vm_readv`, `open`, `pread`, `read`, `write`, `ioctl`, `close`, `usleep`, `signal`, `fcntl`.

Усі програми написані на C++17, зібрані компілятором g++ з прапорцями `-std=gnu++17 -Wall -Wextra -O2`. Код розрахований на Linux; для task2 потрібен `sudo` або членство в групі `input` (альтернативно — тестування через FIFO).